

OPERATING SYSTEMS AND COMPUTER NETWORKS

SIGNAL MANAGEMENT

Lecturer: Milos Antonijevic

SYSTEM CALLS

- When a program invokes a system call, it is interrupted and the system switches to Kernel space. The Kernel then saves the process execution context (so that it can resume the program later) and determines what is being requested. After checking for the validity and privileges, the Kernel processes the request in Kernel Mode and can access the device drivers in charge of controlling the hardware.
- The Kernel can read and modify the data of the calling process as it has access to memory in User Space (e.g. it can copy the keyboard character into a buffer that the calling process has access to).
- When the Kernel is done processing the request, it restores the process execution context that was saved when the system call was invoked, and control returns to the calling program which continues executing.

SIGNALS

- Communication between running programs is called interprocess communication (IPC).

DEFINITION:

Signal is an asynchronous event which is delivered to a process.

- Process can:
 - ignore/discard the signal (not possible with SIGKILL or SIGSTOP)
 - execute a signal handler function, and then possibly resume execution or terminate
 - carry out the default action for that signal
- List of signals: `kill -l` `man 7 signal`

SIGNALS

- Process decides what to do with a signal by declaring or installing a signal handler. A signal handler is a function that is executed when a signal is received.
- Different signal handlers can be executed for different signals.
- Signal handler interrupts the process at its current execution point and execution is jumped to the signal handler function. Each process is given a default signal handler for each signal, usually containing code that terminates the process.
- The **signal()** system call asks the OS to install a signal handler function for a given signal and process.
- Signals can be generated by users, the operating system, or other processes.

INSTALL SIGNAL HANGLER - EXAMPLE

```
#include <stdio.h>
#include <signal.h>
#include <stdlib.h>
main() {
    void f(int); /* prototype for signal handler function */
    int i;
    signal(SIGFPE,f); /* install the handler */
    for (i=-3; i<=3; i++) printf("%d\n",12/i);
}

void f(int signum) {
    printf("You can't divide by zero!\n");
    exit(SIGFPE); /* exits with the information about the reason for termination */
}
```

IGNORE THE SIGNAL - EXAMPLE

```
#include <stdio.h>
#include <signal.h>
main() {
    signal(SIGINT,SIG_IGN); /* tell the OS to ignore the interrupt signal */
    printf("I'm running ...\n");
    while (1) {
        printf("Still going ...\n");
        sleep(1);
    }
}
```

- This program can be terminated by pressing CTRL-\ (SIGQUIT). [[Try using kill command with different signals](#)]

RESTORE DEFAULT HANDLER - EXAMPLE

```
#include <stdio.h>
#include <signal.h>
main() {
    int i;
    signal(SIGINT,SIG_IGN); /* ignore the signal */
    printf("I'm running ...\n");
    for (i=1; i<10; i++) {
        printf("Still going ...\n");
        sleep(1);
        if (i == 3) signal(SIGINT,SIG_DFL); /* re-install default handler */
    }
}
```

KILL() SYSTEM CALL

- Another process can generate a signal by calling the **kill()** system call. Such a signal is usually sent from a parent to a child process, telling the child to terminate.
- However, these signals can be sent in either direction and can be used to provide a limited form of interprocess communication.

```
void f(int signum) {  
    printf("Child killed from parent!\n");  
    running=0;  
}  
...  
signal(SIGUSR1,f);  
kill(i,SIGUSR1);
```